

Captable Multitenancy: Status and Plan (v2)

Date: 2026-10-05 · **Branch:** feat/multitenancy-phase0 (17 commits ahead of main , not pushed) · **Tests:** 193 passing · **Production build:** passes

Replaces: the v1 plan (same path, see git history). v1 mixed a plan with a log and its code samples went stale; this version is status plus remaining work only.

Goal: make tenant isolation structural, enforce roles inside a tenant, and link each company to the TIN Hub's tenant identity without duplicating tenant logic.

Stack: Next.js 14, tRPC 10, Prisma 5.14, NextAuth 4 (JWT), Hono REST, Postgres (Neon), Vitest 1, Biome.

1. Principles (learned the hard way)

- 1. **Inventory the perimeter, not just the routers.** The scoped client protects code that uses it. Most real holes were elsewhere: ids supplied as data, unauthenticated server actions, an empty-session login bypass, pages that query directly. Every entry point is listed in section 3.
- 2. **A guard that is not run is not a guard.** No CI workflow currently runs tests or lint (.github/workflows/* only builds an image). Section 6 fixes that.
- 3. **Tests first, reviewed by a second reader.** Each batch was written test-first and then independently reviewed; the reviews found the three most serious issues.
- 4. **Ownership split (prevents duplicated tenant logic):**

Concern	Owner
Who a person is, SSO, platform operators	Hub
Which tenant a customer is	Hub (Captable stores hubTenantId later)
Membership, role, status in a company	Captable (Member)
What a role may do on cap-table data	Captable (CustomRole , RBAC)
Tenant-scoped data access	Captable (tenantDb)

2. Architecture today

Graph of src/ (graphify, rebuilt today): 534 files · 2,196 nodes · 7,358 edges · 90 communities. withTenant is now a hub node; checkMembership no longer is.

Layer	What it does	Where
tenantDb(db, companyId)	Prisma client extension: forces where.companyId , stamps creates, scopes child models through their parent, rejects raw SQL	src/server/tenant-db.ts
withTenant	tRPC base procedure: session, then one ACTIVE-membership lookup, then ctx.tenant {db, companyId, memberId, role, customRoleId}	src/trpc/api/trpc.ts
withAccessControl	role check on top of withTenant (one membership lookup per request)	same
Guards	assertTenantOwns (ids supplied as data), assertBucketUsable (ownership)	src/server/tenant-guard.ts

Layer	What it does	Where
REST	each auth middleware sets <code>tenantDb</code> from the verified member row	<code>src/server/api/middlewares/*</code>
Uploads	authenticated presign and download procedures; <code>Bucket.companyId</code>	<code>src/trpc/routers/bucket-router</code>
Tests	harness (local DB only), 75 tRPC isolation cases, 26 REST cases, upload tests, architecture guard	<code>tests/tenant/*</code>

Router coverage: 37 files on `withTenant`, 19 on `withAccessControl`, 9 `withAuth` and 8 `withoutAuth` (tenantless or token flows by design).

3. Entry-point inventory

Entry point	Protected by	Status
tRPC routers	<code>withTenant</code> / <code>withAccessControl</code> ; architecture rules (a), (b), (c), (e)	done
REST <code>/v1/{companyId}/...</code>	scoped client in middleware; rule (d); REST tests	done
REST cookie auth	rejects <code>{}</code> and partial sessions	done
Server actions / <code>"use server"</code>	file-uploads no longer a server module	done; page-level directives remain (hygiene)
Server components (4 dashboard pages)	JWT company + global <code>db</code> ; layout re-checks membership	open (S4)
Public token flows (e-sign, data room, updates)	signed token binds the resource	safe by design; tokens never expire
Queue jobs / <code>esign</code> services	trusted server payloads, global <code>db</code>	safe by design
S3 objects	<code>Bucket.companyId</code> , authenticated presign and <code>getUrl</code>	done; legacy NULL-owner buckets unusable

4. Shipped

Commit	What
<code>c421cf8</code> (on <code>main</code> , PR #1)	accept-invite takeover, cross-tenant writes, REST company leak, ACTIVE-member checks
<code>6a07a81</code> , <code>adc0f0a</code> (PR #4, open)	e-sign requires the signed token; Google linking, onboarding email overwrite, <code>NEXTAUTH_SECRET</code> fallback
<code>d3fb476</code> , <code>d2c9c49</code>	test harness with a local-DB guard; cross-tenant matrix
<code>004ef28</code> , <code>21b49f1</code>	<code>tenantDb</code> , <code>withTenant</code> ; architecture guard
<code>25ee605</code>	14 client-supplied-id holes (buckets, recipients, roles, <code>switchCompany</code>)
<code>3790663</code> , <code>19bfa41</code> , <code>aa7b726</code>	all routers moved to the tenant client; <code>revoke-invite</code> fixed
<code>dcf9e3a</code>	REST scoped; three REST holes; route collision (<code>/shares/{id}</code>)
<code>19548a4</code>	empty session authenticated as an arbitrary member (REST cookie path)
<code>556c42e</code>	update editor page cross-tenant read; three missed routers

Commit	What
6889d6c	unauthenticated file upload/download/delete; bucket owner and backfill

Test count by step: 13, 88, 115, 127, 131, 155, 159, 178, 193.

5. Open findings (ranked)

#	Finding	Severity	Phase
F1	No role checks on most cap-table mutations, members toggle/remove/re-invite/revoke, and the new bucket procedures	High	2
F2	4 dashboard pages read tenant data from the JWT company with the global client (stale after deactivation; layout and page render concurrently)	Medium	2
F3	Public-flow tokens (e-sign, data room, update links) never expire	Medium	2
F4	<code>Member.role</code> schema default is <code>ADMIN</code> . Invites set the role explicitly (<code>getRoleById</code>), so it is not exploitable today, but any new create path that omits it creates an admin	Medium	2
F5	<code>getCompanyList</code> and REST <code>company/getMany</code> list memberships of any status	Low	2
F6	Presigned PUTs have no size limit; <code>Bucket</code> has no DB-level relation to <code>Company</code>	Low	3
F7	<code>Stakeholder.email</code> is globally unique (one person cannot be a stakeholder of two companies; leaks existence)	Medium	3
F8	REST: cookie auth fails for requests with a body; header schema demands <code>Authorization</code> for cookie callers; pagination <code>limit</code> default ignored	Low	3
F9	"use server" left on 5 page files (not a hole; hygiene)	Low	3

6. Remaining plan

Scope rule (ponytail): each task is the smallest change with a test that fails first. Tasks marked **D** need a decision from you (section 7).

Phase 2: roles and perimeter (next)

- [] **T1 (F4, check first)**. Confirm every `member.create/upsert` passes a role explicitly; then remove the `@default(ADMIN)` (migration, onboarding passes `ADMIN` for the first member). Test: a member created without a role has no permissions.
- [] **T2 (F1, D2)**. Add RBAC subjects `securities`, `cap-table-settings`, `updates`, `data-rooms`, `templates`; put `.meta({policies})` on every mutation and on `bucket.getUrl/presignUpload/create` (`documents`). Refuse removing, deactivating or demoting the last ADMIN. Test: a CUSTOM role with no grants is denied each mutation (extend the matrix with a "same tenant, insufficient role" column); ADMIN allowed; last-admin refused.
- [] **T3 (F2)**. `getServerTenant()` (cached; `{companyId, db: tenantDb}` from `getPermissions`); use it in the 4 dashboard pages. Add a rule to the architecture test: pages under `src/app/(authenticated)` must not import the global `db`.
- [] **T4 (F3, D)**. Add `exp` to public-flow tokens (e-sign, data room, update link); expired token returns 401. Needs a decision on lifetimes (suggest 30 days, renewable by resend).
- [] **T5 (F5)**. Filter `getCompanyList` and REST `getMany` by ACTIVE, onboarded members.

Phase 3: hygiene

- [] **T6 (F7)**. `Stakeholder` unique on `(companyId, email)`; update callers using the old unique key.

- [] **T7 (F6)**. Add `Bucket -> Company` relation; presign size limit (`Content-Length` condition).
- [] **T8 (F8, F9)**. Fix REST cookie auth with a body, header schema, pagination default; drop the page-level `"use server"` directives.

Phase 4: Hub linkage (gated, not scheduled)

Opens only when the Hubs provide: a published signing key (JWKS), a claims contract with `tenant_id`, and a membership read endpoint or webhook (none exist today), plus decision D3. Then: an OIDC or WorkOS provider mapping IdP organisation to `Company.hubTenantId`; a signed membership webhook; operator access through a Hub claim only. **Decision:** the `hubTenantId` / `hubUserId` columns (old Task 13) are dropped from this plan until then; unused columns are speculative.

Phase 5: row-level security (deferred)

Trigger: a second isolation bug found despite `tenantDb`, or a compliance need. Approach is a documented Prisma pattern: a client extension that runs `set_config('app.current_company_id', ...)` in a transaction with each query, plus `CREATE POLICY ... USING (companyId = current_setting(...))` on the 18 tenant tables.

Rollout and operations (before and after merge)

1. **Merge PR #4**, then rebase this branch onto `main` and open one PR (about 80 files; split by phase if reviewers prefer). **Every PR targets `main`** (the stacked PRs landed in the wrong branch once).
2. **Database:** the `Bucket.companyId` migration is applied only to the local test DB. Before any database that holds files: run `docs/bucket-owner-backfill-report.sql` (read-only) and review the buckets that would stay unowned. Neon is empty, so applying there is trivial. Production migration is run by a person with credentials, never delegated.
3. **Config:** `NEXTAUTH_SECRET` must be 32+ characters; the app now refuses to start otherwise.
4. **Behaviour changes to announce:** non-members get UNAUTHORIZED instead of `{success:false}`; REST share update is now `PATCH /v1/{companyId}/shares/{id}`; Google sign-in no longer merges into an existing password account.
5. **CI (new):** add a workflow job that starts a Postgres service, runs `prisma migrate deploy` against it, then `pnpm test`, `tsc --noEmit` and `biome check`. Without it the isolation tests protect nobody.
6. **Smoke test (manual, once per release):** upload and open a document; sign a document through an emailed link; invite and accept a member; switch company; open a data room link. These flows are covered by tests and the build, never by a browser run.

7. Decisions needed

#	Decision	Default in this plan	Blocks
D1	Stay on NextAuth and Prisma; link to the Hub by tenant id	Stay	Phase 4
D2	Role matrix for CUSTOM roles on cap-table subjects	Read for all members; writes need a grant; member management admin-only	T2
D3	Which login provider the Hub fronts (Supabase Auth or WorkOS)	Unknown	Phase 4
D4	Public-link token lifetime	30 days	T4
D5	One large PR or one per phase	One per phase	Rollout

8. Verified against current documentation (Context7)

Claim the plan relies on	Source says	Effect on plan
Prisma treats <code>undefined</code> in a filter as "do nothing" (key omitted)	Prisma docs, null vs undefined	Root cause of the empty-session bypass; explains why <code>checkMembership</code> now rejects falsy ids
Query extensions via <code>\$allModels.\$allOperations</code> can rewrite <code>args</code> ; raw queries reach <code>\$allOperations</code> with <code>model</code> undefined	Prisma client extensions docs	<code>tenantDb</code> design; raw SQL is rejected explicitly
RLS with Prisma: <code>set_config(...)</code> per query inside a transaction, plus policies, is a documented pattern	Prisma client extensions blog	Phase 5 approach
"use server" exports are public endpoints; authenticate and authorize inside each action	Next.js 14 Server Actions and authentication docs	The upload fix (no unauthenticated server actions)
<code>/api/auth/session</code> returns an empty object when there is no session; <code>getSession</code> returns <code>null</code>	NextAuth.js (v4) docs	The cookie path must treat <code>{}</code> as unauthenticated
tRPC middleware extends context with <code>next({ ctx })</code> ; <code>createCaller</code> is for server-side calls and tests, not for calling procedures from other procedures	tRPC server docs	<code>withTenant</code> and the test harness

The Context7 results include Prisma v7 pages while this repo is on 5.14; behaviours above were also confirmed by the test suite on the installed versions.

9. Model routing (remaining work)

Task	Model	Why
T1, T2, T3, T4	<code>opus</code>	Auth, roles, tokens: security paths
T5, T6, T7, T8	<code>sonnet</code>	Mechanical with a failing test; <code>opus</code> review per phase
CI job, smoke-test checklist	<code>sonnet</code>	Config
Migrations on shared databases, deploys	<code>orchestrator</code>	Credentials; never delegated

Each phase ends with an independent `opus` review of the diff (the reviews found the empty-session bypass, the open editor page and the weak backfill) and a written summary of the previous phase before the next starts.

10. Hub dependencies (not Captable work)

Repo	Needed before Phase 4
<code>shared-identity-hub</code>	Close the <code>x-tenant-id</code> header bypass in <code>get_current_tenant_id()</code> ; one definition of "operator" (four exist); tenant switching and the Next adapter (specified, not built)
<code>tin-boss-api</code>	Tenant and membership read endpoint or webhook; JWKS endpoint; claims revocation
<code>shared-client-care-hub</code>	Composite <code>(tenant_id, id)</code> keys on CRM tables; one tenant-resolution function (not needed by Captable)